

Table of contents

1	Introducción	1
2	Objetivos	2
3	Descripción de la práctica	2
3.1	Herramientas	2
3.2	Guías	3
4	Desarrollo	3
4.1	Fase 1 — Repositorio	3
4.2	Fase 2 — Librería	4
4.3	Fase 3 — Cuaderno	5
5	Requisitos para la entrega	5
5.1	Uso de herramientas de IA	6
6	Entrega	6
7	Evaluación	6
8	Notas	7
9	Extra (hasta +2 puntos)	7
9.1	Gestión avanzada del repositorio	7
9.2	Buen rendimiento del modelo	7
9.3	Independencia del dataset	7
9.4	Independencia de la página web	7

1 Introducción

En esta práctica, te convertirás en un ingeniero lingüístico y te enfrentas a un reto: **el Esperanto**. Esperanto es la lengua artificial más hablada del mundo, pero apenas cuenta con herramientas de Procesamiento de Lenguaje Natural. Tu misión es construir, de principio a fin, un sistema capaz de **clasificar textos en Esperanto por temática**: desde la recopilación de datos, pasando por tu propia librería de NLP, hasta la clasificación de páginas web reales. Para ello usarás técnicas de web scraping, consumo de APIs, NLP y aprendizaje automático con Python.

La práctica se llevará a cabo en **equipos de máximo 3 personas** (preferiblemente 3). Cada equipo elegirá **categorías temáticas** sobre las que aprender a clasificar textos.

2 Objetivos

- Poner en práctica el temario de la asignatura **Minería de Textos**: NLP y minería web.
- Formalizar conocimiento lingüístico de forma que admita un tratamiento algorítmico (CE25).
- Aplicar técnicas de aprendizaje computacional para la extracción automática de información a partir de grandes volúmenes de datos (CE26).
- Adquirir experiencia con flujos de trabajo profesionales: repositorios, librerías, documentación y pruebas.

3 Descripción de la práctica

La práctica se realizará en equipos de máximo 3 personas. Se elegirán **categorías temáticas** validadas por el profesor y se construirá un **dataset**, una **librería NLP** y un cuaderno con un **modelo de clasificación** capaz de clasificar textos en Esperanto por temática. Todo esto se desarrollará en **Python** y se alojará en **GitHub**.

La práctica se dividirá en 3 fases:

1. **Fase 1 — Repositorio:** Gestión del repositorio y construcción del dataset.
2. **Fase 2 - Librería:** Desarrollo de la librería de NLP.
3. **Fase 2 — Cuaderno:** Entrenamiento del modelo de clasificación y clasificación de páginas web.

Se ha elegido el Esperanto de forma deliberada. Al ser una lengua artificial, con reglas gramaticales regulares y sin excepciones, permite construir herramientas lingüísticas propias de manera abordable.

3.1 Herramientas

Estas son las herramientas principales que se usarán en la práctica:

- **GitHub:** el equipo creará un repositorio público en [GitHub](#) donde alojará todo su trabajo utilizando metodologías Open Source y `git`.
- La **librería de NLP**, así como cualquier otro script, se desarrollarán en [Python](#).
- El **entrenamiento del modelo** y la **clasificación web** se desarrollarán en un único cuaderno Jupyter de Python (`.ipynb`).
 - Se aconseja comprobar su funcionamiento sobre [Google Colab](#).
- Se permite el uso de cualquier librería vista en clase (`requests`, `beautifulsoup4`, `pandas`, `scikit-learn`, `matplotlib`...).
- Se permite y aconseja el uso de herramientas de IA (Gemini u otras) como apoyo. Estas herramientas nunca deben usarse en sustitución del alumno. Ver [Uso de herramientas de IA](#) para más detalles.

3.2 Guías

Como guía, se proporciona el siguiente repositorio de ejemplo: <https://github.com/jparisu/nlp-esperantilo>. Este repositorio cuenta con:

- Una primera versión muy básica de librería de NLP en Esperanto. Esta se puede usar como referencia para construir la librería del equipo.
 - TODO LINK
- Una guía con toda la información técnica necesaria para llevar a cabo la práctica. Esta información complementa el temario de la asignatura con información sobre:
 - **git**: TODO LINK
 - **GitHub**: TODO LINK
 - **Librería de Python**: TODO LINK
 - **Esperanto**: TODO LINK
- Un script para la recopilación de datos desde la **API de Wikipedia**. Este script puede usarse como ayuda para construir el dataset del equipo, aunque se permite usar cualquier otra fuente de datos.
 - TODO LINK

4 Desarrollo

El proyecto se divide en dos fases, que se desarrollarán de forma secuencial:

4.1 Fase 1 — Repositorio

En esta primera fase, el equipo creará un repositorio GitHub, gestionándolo correctamente y construyendo el dataset de entrenamiento.

4.1.1 Selección de temáticas

Cada equipo deberá elegir al menos **3 categorías temáticas** sobre las que clasificar textos en Esperanto. Estas categorías deben ser **semánticamente cercanas** para que el problema de clasificación no sea trivial. El profesor validará la elección de las categorías y podrá sugerir cambios si lo considera necesario.

Tip

Antes de elegir las temáticas, se recomienda explorar las distintas posibilidades de obtención de datos y comprobar que son viables.

Estos son algunos ejemplos de categorías temáticas que se pueden elegir:

- Deportes: fútbol, baloncesto, tenis.
- Ciencia: física, biología, química.
- Historia: Edad Media, Renacimiento, Edad Contemporánea.
- Geografía: ríos, mares, lagos.
- Cultura: cine, música, literatura.

- Noticias: política, economía, deportes.
- Gastronomía: cocina, bebidas, postres.
- Animales: mamíferos, aves, reptiles.
- Dinosaurios: terópodos, saurópodos, ornitópodos.
- Personajes: históricos, literarios, contemporáneos.
- etc.

4.1.2 Gestión del repositorio

Se creará y gestionará un repositorio de [GitHub](#) que albergará el *script* de recogida de datos, el dataset, la librería y el cuaderno de entrenamiento y clasificación.

El repositorio debe reflejar un uso ordenado del control de versiones: estructura clara, *commits* significativos y contribución equilibrada de todos los integrantes.

Resultado: un repositorio de GitHub organizado y documentado.

4.1.3 Construcción del dataset

Se construirá un *script* que recupere texto de las categorías elegidas, lo divida en párrafos y genere el dataset etiquetado conforme al *Contrato de datos*.

Heredar la etiqueta del artículo introduce **ruido**: hay párrafos genéricos que no distinguen la categoría (p.ej. “*Ĝi estis fondita en 1950*”). Reconocer, comentar y, en la medida de lo posible, medir ese ruido forma parte del trabajo.

Resultado: un dataset etiquetado en Esperanto (`text;class`) y el *script* reproducible que lo genera, alojados en GitHub.

4.1.3.1 API de Wikipedia

Se proporciona una **función** de ejemplo que recupera texto de la **API de Wikipedia** en Esperanto. Esta se puede usar en el script para construir el dataset a partir de artículos de Wikipedia.

No es obligatorio usar dicho API, se puede usar cualquier otra fuente de datos. Sin embargo, es recomendable dada la cantidad de contenido disponible en Esperanto y la facilidad de uso de la API.

4.2 Fase 2 — Librería

En esta segunda fase, el equipo desarrollará una librería de NLP en Esperanto con una **API similar a la de spaCy**. La librería debe ir acompañada de tests y documentación.

4.2.1 Librería de NLP en Esperanto

Se desarrollará una librería de Python de NLP para Esperanto con una **API similar a la de spaCy**, aprovechando la regularidad gramatical de la lengua. La librería debe ser **basada en reglas** e incluir, como mínimo:

- Tokenizador.
- Lematizador.
- Etiquetador morfológico / gramatical (POS).
- Gestión de *stop-words*.
- Objetos tipo `Doc` / `Token` que estructuren el resultado.

Resultado: una librería de Python instalable, con documentación y pruebas.

4.3 Fase 3 — Cuaderno

En esta fase se construirá un cuaderno Jupyter (`.ipynb`) que use el dataset y la librería para entrenar un modelo de clasificación y clasificar páginas web reales.

4.3.1 Entrenamiento del modelo de clasificación

Usando el dataset de la Fase 1 y la librería de la Fase 2, se construirá una representación **tabular** del corpus con técnicas vistas en clase y se entrenará un modelo de clasificación. El algoritmo es de libre elección, siempre que su entrenamiento y uso no requieran grandes recursos computacionales.

Debe seguirse la metodología estándar de aprendizaje automático: preprocesamiento, partición *train/test* o validación cruzada, y evaluación con métricas adecuadas (*accuracy*, F1 macro y matriz de confusión).

Resultado: un modelo entrenado, junto con su evaluación de rendimiento, en un cuaderno Jupyter ejecutable en Google Colab.

4.3.2 Clasificación de páginas web

Para testear el modelo entrenado en un entorno real, se elegirá una página web en Esperanto que contenga texto de las categorías elegidas. Mediante web scraping, se extraerá el texto de una página web y se determinará, con el modelo entrenado, la probabilidad de pertenecer a cada categoría.

Resultado: el mismo cuaderno, ampliado con la obtención y clasificación de texto web.

5 Requisitos para la entrega

- El dataset y la librería se alojarán en un repositorio de Github público y se descargarán automáticamente desde el cuaderno.
- El cuaderno debe estar desarrollado en un fichero `.ipynb` y ser ejecutable en Google Colab.
- El cuaderno debe ser **autocontenido**: no debe depender de ficheros locales ni de interacción del usuario.

- El código debe ejecutarse de forma autónoma y ser capaz de sobrevivir a un fallo en cualquiera de las webs utilizadas.
- Para el desarrollo se deberán usar técnicas de minería web y técnicas de NLP vistas en clase.

5.1 Uso de herramientas de IA

Esta práctica contempla un proyecto complejo de desarrollo de software con múltiples fases y entregables. En esta práctica, el enfoque del alumno debe ser en el entendimiento de los conceptos y en la capacidad de razonar y diseñar un sistema complejo, trasladando las tareas de menor valor añadido a herramientas de IA. Por esta razón, el uso de herramientas de IA es aconsejable para las siguientes tareas:

- **Guía técnica:** búsqueda de información, resolución de dudas, ejemplos de código, etc.
- **Código fuente:** escritura de código fuente, tests, documentación de código, etc.
- **Tareas tediosas:** escritura de documentación, resolución de errores, etc.

Se pueden usar tanto herramientas chatbot como agentes de IA **habilitadas por la universidad**. No se pueden usar IAs de pago fuera de las licencias facilitadas por la universidad.

! Important

Estas herramientas de IA nunca podrán ser sustitutas del alumno. El alumno debe ser capaz de **explicar y razonar** cualquier decisión de diseño o resultado obtenido.

6 Entrega

La entrega tiene dos hitos:

- **Punto de control (mitad de la práctica):** enlace al repositorio de GitHub con el *script*, el dataset y la librería (Fase 1 y 2).
 - Fecha: *se indicará en Canvas*.
- **Entrega final:** cuaderno `.ipynb` con el código y los resultados (Fase 2).
 - Fecha: *se indicará en Canvas*.

7 Evaluación

La práctica se defenderá **por todo el equipo** ante el profesor. El equipo deberá explicar el desarrollo, las decisiones de diseño y los resultados, responder a preguntas sobre el código y ser capaz de realizar pequeñas modificaciones que se soliciten en el momento. La calificación del equipo es **compartida**.

La práctica se puntúa del siguiente modo (sobre 10):

- Gestión del repositorio — **1 punto**
- Construcción del dataset — **1,5 puntos**
- Librería de NLP — **3,5 puntos**
- Modelo de clasificación — **2,5 puntos**
- Clasificación de páginas web — **1,5 puntos**

8 Notas

- Todos los integrantes deben asistir a la defensa. Quien no asista obtendrá un 0 en la práctica.
- Los resultados deben ser reproducibles: fijar semillas aleatorias y versiones de las APIs y librerías utilizadas.
- Si el tiempo y las circunstancias lo permiten, y con el consentimiento del equipo, la práctica podrá defenderse en clase frente al resto de compañeros, teniéndose en cuenta positivamente para la nota de participación.
- La nota máxima de la práctica es de 10 puntos.

9 Extra (hasta +2 puntos)

Existen 4 apartados adicionales que sumarán hasta **0.5 puntos** cada uno en caso de resolverse correctamente.

9.1 Gestión avanzada del repositorio

El repositorio debe contar con una página *Read the Docs* con la documentación de la librería y un flujo de trabajo de *GitHub Actions* que ejecute automáticamente las pruebas unitarias de la librería cada vez que se haga un *push* al repositorio.

9.2 Buen rendimiento del modelo

El modelo de clasificación debe obtener resultados sólidos, bien justificados. Realizar un análisis exhaustivo del modelo entrenado, con métricas, visualizaciones, ejemplos de predicciones correctas e incorrectas, y conclusiones.

9.3 Independencia del dataset

El dataset del equipo debe cumplir el *Contrato de datos*: **CSV** con codificación **UTF-8**, separador **;**, dos columnas con cabecera **text;class**, granularidad de párrafo y categorías semánticamente cercanas. El cuaderno debe ser capaz de reentrenar y evaluar el modelo de clasificación sobre el dataset de cualquier otro equipo que cumpla dicho contrato.

9.4 Independencia de la página web

El cuaderno debe ser capaz de clasificar correctamente el texto de cualquier página web proporcionada, sin depender de la estructura de las webs utilizadas durante el desarrollo.